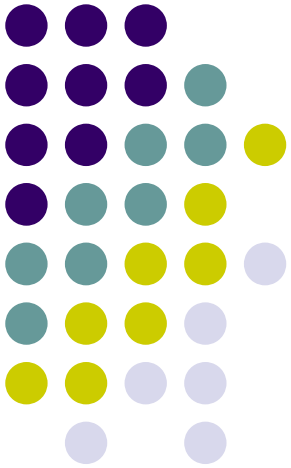
Acceso a bases de datos en Java

JDBC 3.0

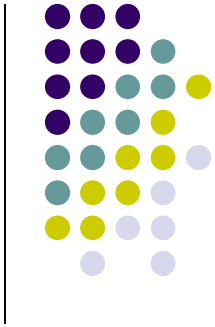
Micael Gallego

micael.gallego@gmail.com



Acceso a bases de datos en Java

JDBC



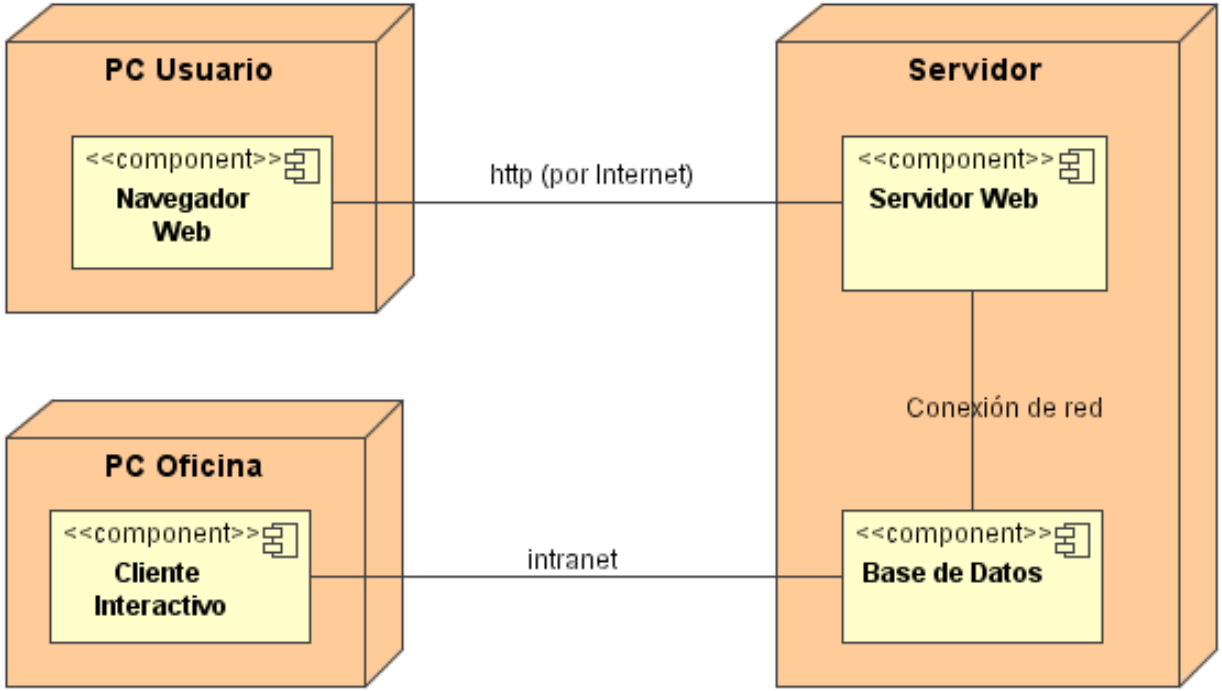
- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

Bases de datos relacionales

- Bases de datos (Database systems)
  - Son programas utilizados para almacenar información y permitir un acceso posterior a ella
  - Varios programas (servidor web, aplicación gráfica, …) pueden acceder a la información de forma concurrente a través de la red
  - La información está centralizada, actualizada y es más sencillo realizar actualizaciones y copias de seguridad
  - La información puede estar en forma de texto, números, ficheros, XML, etc...
  - Existen muchos tipos de bases de datos, pero las más usadas son las bases de datos **relacionales**

Bases de datos relacionales

- Arquitectura



Bases de datos relacionales

- Una base de datos **relacional** almacena la información en tablas* con filas y columnas (campo)

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Tabla Autores

idAutor	nombre	nacionalidad
1	Antonio	Español
2	Gerard	Frances

Tabla RelacionLibroAutor

idLibro	idAutor
1	1
2	2
3	2

* A las tablas se las denominaba “relaciones”, de ahí el nombre de base de datos relacional

Bases de datos relacionales

- Una base de datos **relacional** almacena la información en tablas* con filas y columnas (campo)

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Tabla Autores

idAutor	nombre	nacionalidad
1	Antonio	Español
2	Gerard	Frances

Tabla RelacionLibroAutor

idLibro	idAutor
1	1
2	2
3	2

La información se relaciona mediante identificadores (id)

* A las tablas se las denominaba “relaciones”, de ahí el nombre de base de datos relacional

Bases de datos relacionales

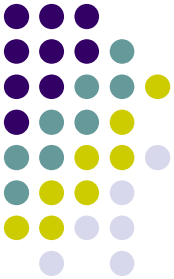
- SQL (*Standard Query Language*): Lenguaje de consulta estándar que permite:
  - Consulta de los datos seleccionando con diferentes criterios y realizando operaciones (medias, sumas, …)
  - Inserción, Actualización y borrado de la información
  - Creación, alteración y borrado de las tablas y sus campos
  - Gestión de usuarios y sus privilegios de acceso

Bases de datos relacionales

- Sus datos cumplen reglas de integridad:
  - Las filas no tienen una posición concreta
  - Un campo no puede ser una lista de valores
  - No existen filas con todos los campos iguales (las haría indistinguibles y podría provocar problemas)
  - Existe al menos una columna que identifica a la fila (id), denominada clave primaria (Primary Key)
  - Se permiten valores NULL diferentes del 0 o de la cadena de caracteres vacía (para expresar dato no disponible)

Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

SQL

- SQL es un lenguaje diseñado para gestionar una base de datos relacional
- Existen 3 versiones del estándar (SQL-92, SQL-99 y SQL-2003)
- El soporte del estándar es muy variado
  - Algunas bases de datos soportan subconjuntos
  - Otras bases de datos soportan extensiones propietarias
  - Cada base de datos tiene un manual con los elementos de SQL que soporta

SQL
Comandos

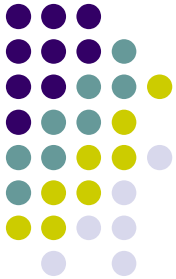
- Los comandos SQL se dividen en categorías:
  - Lenguaje de Manipulación de datos (DML)
    - Obtiene, Inserta, Borra y actualiza datos
    - SELECT, INSERT, DELETE, UPDATE
  - Lenguaje de Definición de datos (DDL)
    - Crea, borra y cambia tablas, usuarios, vistas, índices…
    - CREATE TABLE, DROP TABLE, ALTER TABLE

SQL

Sentencia SELECT

- También conocido como *statement* o *query* (consulta)
- Permite recuperar la información de una o varias tablas
- Especifica uno o más campos, una o más tablas y un criterio de selección
- La base de datos devuelve los campos indicados de aquellas filas que cumplan el criterio de selección

SQL
Sentencia SELECT



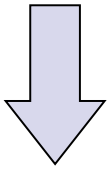
Situación en la base de datos

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Consulta

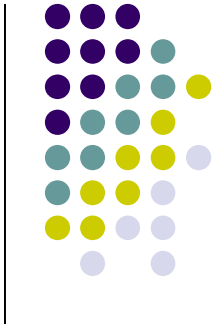
```
SELECT titulo, precio
FROM Libros
WHERE precio > 2
```



Conjunto de resultados (*ResultSet*)

titulo	precio
Bambi	3
Batman	4

SQL
Sentencia SELECT



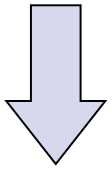
Situación en la base de datos

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Consulta que devuelve todas las filas de la tabla

```
SELECT *
FROM Libros
```



Conjunto de resultados (*ResultSet*)

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

SQL
Claúsula WHERE

- Operador LIKE (Comparación de cadenas)

```
SELECT titulo, precio
FROM Libros
WHERE titulo LIKE 'Ba%'
```

- Operadores relacionales (<,=,…) lógicos (AND, OR)

```
SELECT titulo, precio
FROM Libros
WHERE precio > 3 AND titulo LIKE '%Man'
```

SQL
Uniones (*JOINS*)

- Se pueden unir varias tablas en una consulta

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Tabla Autores

idAutor	nombre	nacionalidad
1	Antonio	Español
2	Gerard	Frances

Tabla RelacionLibroAutor

idLibro	idAutor
1	1
2	2
3	2

SQL
Uniones (*JOINS*)

- Se pueden unir varias tablas en una consulta

Tabla Libros

idLibro	titulo	precio
1	Bambi	3
2	Batman	4
3	Spiderman	2

Tabla Autores

idAutor	nombre	nacionalidad
1	Antonio	Español
2	Gerard	Frances

Tabla RelacionLibroAutor

idLibro	idAutor
1	1
2	2
3	2

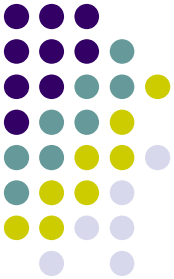
SQL
Uniones (*JOINS*)

```
SELECT titulo, precio, nombre
FROM Libros, Autores, RelacionLibroAutor
WHERE Libros.idLibro = RelacionLibroAutor.idLibro
   AND Autores.idAutor = RelacionLibroAutor.idAutor
```

titulo	precio	nombre
Bambi	3	Antonio
Batman	4	Gerard
Spiderman	2	Gerard

Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

Transacciones

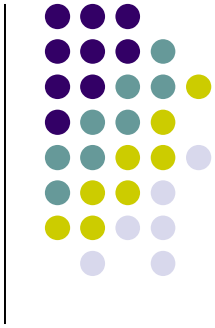
- Varios usuarios pueden acceder a la misma base de datos al mismo tiempo
- Si un usuario está actualizando un conjunto de datos mientras el otro usuario está realizando una consulta, el segundo usuario podría obtener datos parcialmente actualizados (inconsistentes)
- Las transacciones permiten mantener los datos en un estado consistente mientras más de un usuario accede a los datos a la vez

Transacciones

- Una transacción es un conjunto de sentencias SQL que forman una unidad lógica y deben ejecutarse todas para mantener la integridad de los datos
- Si alguna de las sentencias falla, se deshace todo lo que hayan cambiado las demás
- El usuario finaliza una transacción con un comando **commit**
- Si quiere cancelar una transacción que no ha finalizado y deshacer todos los cambios, puede usar el comando **rollback**
- Si existen problemas de concurrencia con varios usuarios, la propia base de datos puede deshacer la transacción (con un **rollback**)

Acceso a bases de datos en Java

JDBC



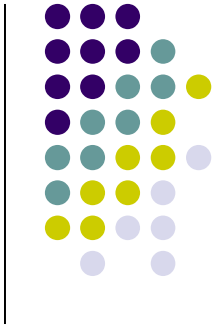
- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

Procedimientos Almacenados

- Un procedimiento almacenado es un grupo de sentencias SQL que pueden ser llamadas por su nombre
- Admiten parámetros
- Se almacenan y se ejecutan en la base de datos
- Algunos productos tienen un lenguaje imperativo (PL/SQL de Oracle) y otros permiten implementarlos en Java

Acceso a bases de datos en Java

JDBC



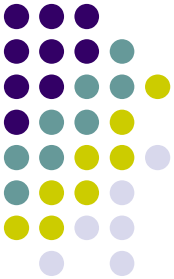
- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

Metadatos

- Las bases de datos tienen tablas de sistema (*system tables*) que guardan información sobre las tablas de usuario
  - Nombre de las tablas
  - Campos de las tablas (nombre, tipo, …)
- A la información sobre las tablas se la denomina **Metadatos**
- Suele usarse en aplicaciones que permiten el acceso a una base de datos cualquiera
  - Generadores de informes
  - Clientes interactivos de acceso a base de datos

Acceso a bases de datos en Java

JDBC



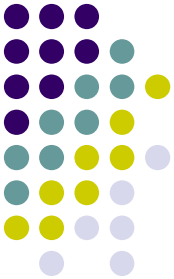
- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

Productos

- Existen muchos productos de bases de datos, comerciales y software libre
  - MySQL (Software Libre) – http://www.mysql.org
  - Derby (Software Libre) - http://db.apache.org/derby
  - Oracle (Comercial) - http://www.oracle.com
  - MS SQL Server (Comercial) - http://www.microsoft.com/sql

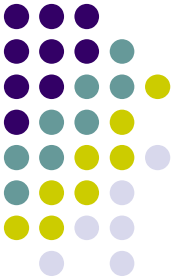
Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
  - Bases de datos relacionales
    - SQL
    - Transacciones
    - Procedimientos Almacenados
    - Metadatos
  - Productos
  - MySQL
- Bases de datos en Java. JDBC

MySQL



- http://www.mysql.org/
- Sistema gestor de base de datos multiplataforma
- Desarrollado en C
- Licencia código abierto GPL
- Soporte de un subconjunto de SQL 99
- Dispone de driver JDBC para Java
- Herramienta interactiva para hacer consultas y crear bases de datos

MySQL

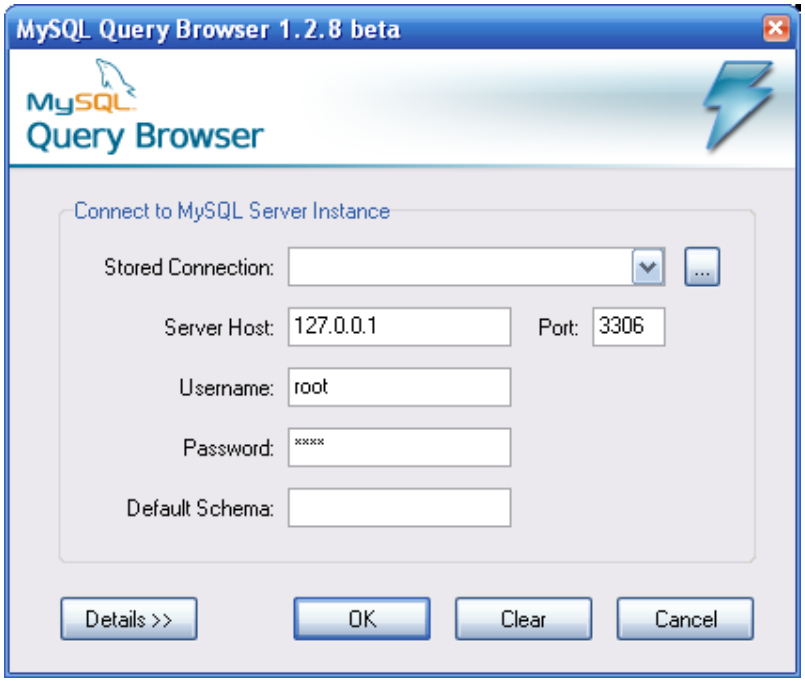
- Instalación
  - Servidor de Base de Datos:
    - MySQL 5.0 Database Server - Community Edition
    - mysql-5.0.41-win32.exe
  - Herramientas interactivas:
    - MySQL GUI Tools
    - mysql-gui-tools-5.0-r12-win32.msi
  - Driver JDBC:
    - MySQL Connector/J
    - mysql-connector-java-5.0.6.zip

MySQL

- Instalación del Servidor de Base de Datos
  - Seguir las opciones por defecto
  - Asignar contraseña al root, por ejemplo “*pass*”

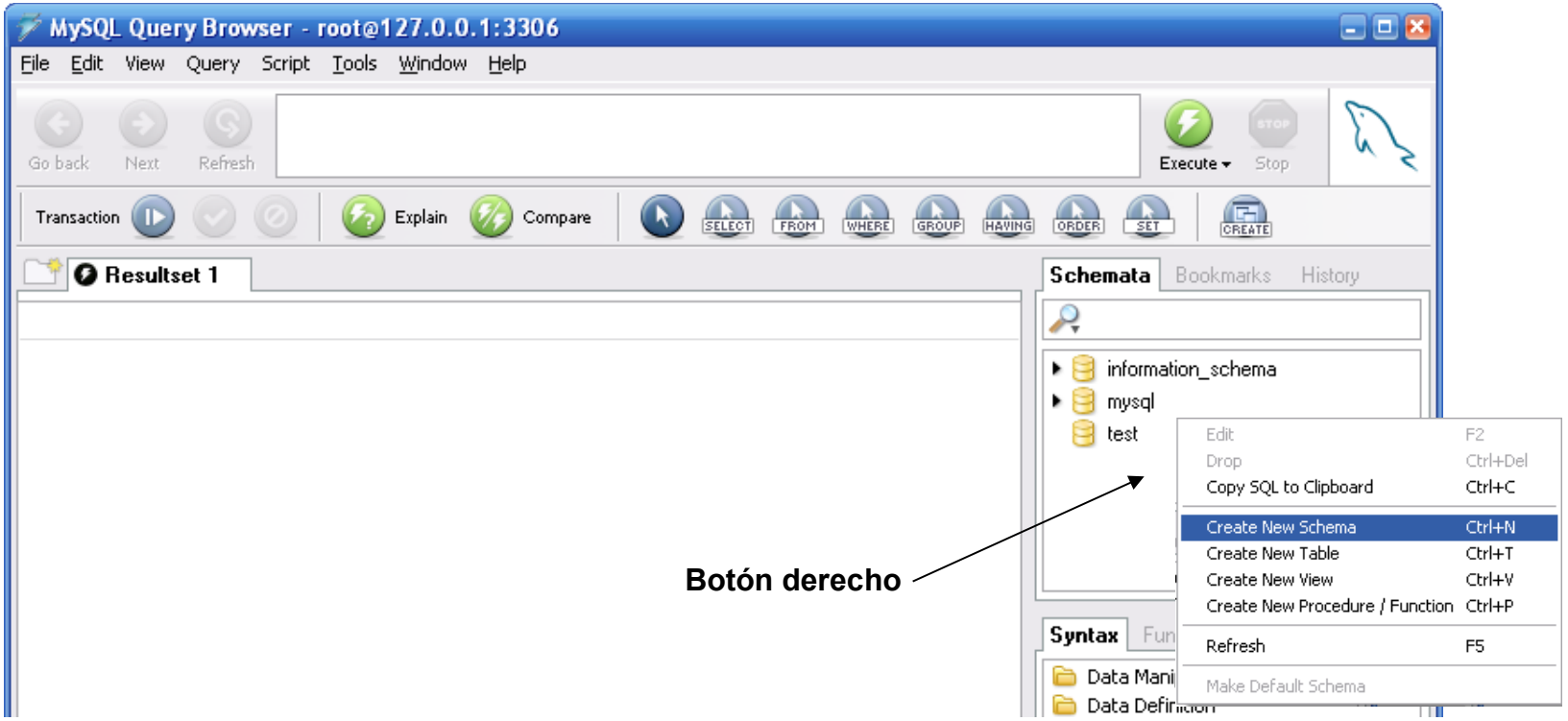
MySQL

- Instalación de las herramientas interactivas
  - Seguir las opciones por defecto
  - Iniciar MySQL Query Browser



MySQL

- Creamos el esquema “sample” (Create New Schema) para los ejemplos del curso
- Le ponemos como esquema por defecto (Make Default Schema)



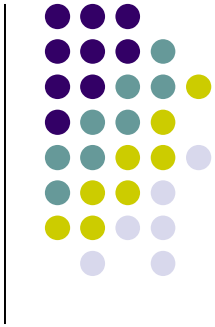
MySQL

- Creamos tablas
  - CREATE TABLE Libros (idLibro INT NOT NULL, titulo VARCHAR(30), precio INT NOT NULL, CONSTRAINT claveLibro PRIMARY KEY (idLibro))
  - CREATE TABLE Autores (idAutor INT NOT NULL, nombre VARCHAR(30), nacionalidad VARCHAR(20), CONSTRAINT claveAutor PRIMARY KEY (idAutor))
  - CREATE TABLE RelacionLibroAutor (idLibro INT, idAutor INT, CONSTRAINT claveRelacion PRIMARY KEY (idLibro, idAutor), CONSTRAINT claveLibroForanea FOREIGN KEY (idLibro) REFERENCES libros(idLibro), CONSTRAINT claveAutorForanea FOREIGN KEY (idAutor) REFERENCES autores(idAutor))

MySQL

- Insertamos valores de ejemplo
  - INSERT INTO Libros VALUES (1, 'Bambi', 3)
  - INSERT INTO Libros VALUES (2, 'Batman', 4)
  - INSERT INTO Libros VALUES (3, 'Spiderman', 2)
  - INSERT INTO Autores VALUES(1, 'Antonio', 'Español')
  - INSERT INTO Autores VALUES(2, 'Gerard', 'Frances')
  - INSERT INTO RelacionLibroAutor VALUES (1,1)
  - INSERT INTO RelacionLibroAutor VALUES (2,2)
  - INSERT INTO RelacionLibroAutor VALUES (3,2)
- Consultas de ejemplo
  - SELECT titulo, precio FROM Libros WHERE precio > 2

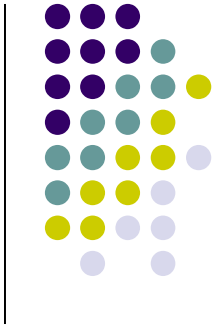
MySQL



- Instalación del driver JDBC
  - Descomprimimos el fichero mysql-connector-java-5.0.6.zip
  - Añadimos la librería mysql-connector-java-5.0.6-bin.jar
    - Si compilamos desde línea de comandos, añadimos el fichero a la variable de sistema CLASSPATH
    - Si usamos Eclipse, Project > Properties > Java Build Path > Libraries > Add External JARs…
  - Driver: **`com.mysql.jdbc.Driver`**
  - URL: **`jdbc:mysql://localhost:3306/sample`**

Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de **`ResultSet`**

Introducción a JDBC

- *Java DataBase Connectivity*
- Es la API (librería) estándar de acceso a base de datos desde Java
- Está incluida en Java SE (*Standard Edition*)
- En Java SE 6 se incluye JDBC 4.0, pero actualmente la mayoría de bases de datos soportan JDBC 3.0
- Más información
  - http://java.sun.com/javase/technologies/database
  - http://java.sun.com/docs/books/tutorial/jdbc/

Introducción a JDBC

- Para conectarse a una base de datos concreta, es necesario su driver JDBC
- El driver es un fichero JAR que se añade a la aplicación como cualquier otra librería (no necesita instalación adicional)
- La mayoría de las bases de datos incorporan un driver JDBC
- ODBC (*Open DataBase Connectivity*) es un estándar de acceso a base de datos desarrollado por Microsoft. Sun ha desarrollado un driver que hace de puente entre JDBC y ODBC aunque no suele usarse.

Introducción a JDBC

- Los pasos para que una aplicación se comunique con una base de datos son:
  1. Cargar el driver necesario para comprender el protocolo que usa la base de datos concreta
  2. Establecer una conexión con la base de datos, normalmente a través de red
  3. Enviar consultas SQL y procesar el resultado
  4. Liberar los recursos al terminar
  5. Manejar los errores que se puedan producir

```java
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close();
        stmt.close();
        conn.close();
  }
}
```

```
Bambi          3.0
Batman         4.0
```

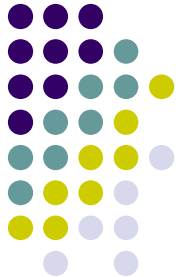
Ejercicio 1

- Implementar el ejemplo anterior
- Comprobar su funcionamiento
- En las siguientes transparencias se explicará en detalle el significado de cada una sus partes

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

       Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Carga del driver

Carga del driver



- Antes de poder conectarse a la base de datos es necesario cargar el driver JDBC
- Sólo hay que hacerlo una única vez al comienzo de la aplicación

```
Class.forName("com.mysql.jdbc.Driver");
```

- El nombre del driver debe venir especificado en la documentación de la base de datos
- Se puede elevar la excepción **`ClassNotFoundException`** si hay un error en el nombre del driver o si el fichero .jar no está correctamente en el **`CLASSPATH`** o en el proyecto

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Establecer una conexión

Introducción a JDBC

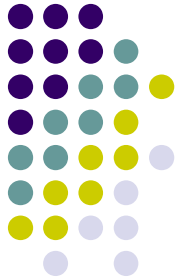
Establecer una conexión

- Las bases de datos actúan como servidores y las aplicaciones como clientes que se comunican a través de la red
- Un objeto **`Connection`** representa una conexión física entre el cliente y el servidor
- Para crear una conexión se usa la clase **`DriverManager`**
- Se especifica la URL, el nombre y la contraseña

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/sample","root","pass");
```

Introducción a JDBC

Establecer una conexión



- El formato de la URL debe especificarse en el manual de la base de datos
- Ejemplo de MySQL

```
jdbc:mysql://<host>:<puerto>/<esquema>
```

```
jdbc:mysql://localhost:3306/sample
```

- El nombre de usuario y la contraseña dependen también de la base de datos

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Ejecutar una sentencia SQL

Ejecutar una sentencia SQL

- Una vez que tienes una conexión puedes ejecutar sentencias SQL
- Primero se crea el objeto **`Statement`** desde la conexión
- Posteriormente se ejecuta la consulta y su resultado se devuelve como un **`ResultSet`**

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
   "SELECT titulo, precio FROM Libros WHERE precio > 2");
```

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Acceso al conjunto de resultados

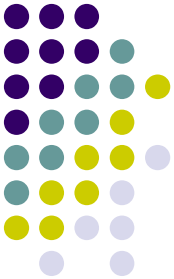
Acceso al conjunto de resultados

- El **`ResultSet`** es el objeto que representa el resultado
- No carga toda la información en memoria
- Internamente tiene un cursor que apunta a un fila concreta del resultado en la base de datos
- Hay que posicionar el cursor en cada fila y obtener la información de la misma

```
while (rs.next()) {
   String name = rs.getString("titulo");
   float price = rs.getFloat("precio");
   System.out.println(name + "\t" + price);
}
```

Acceso al conjunto de resultados

Posicionamiento del cursor

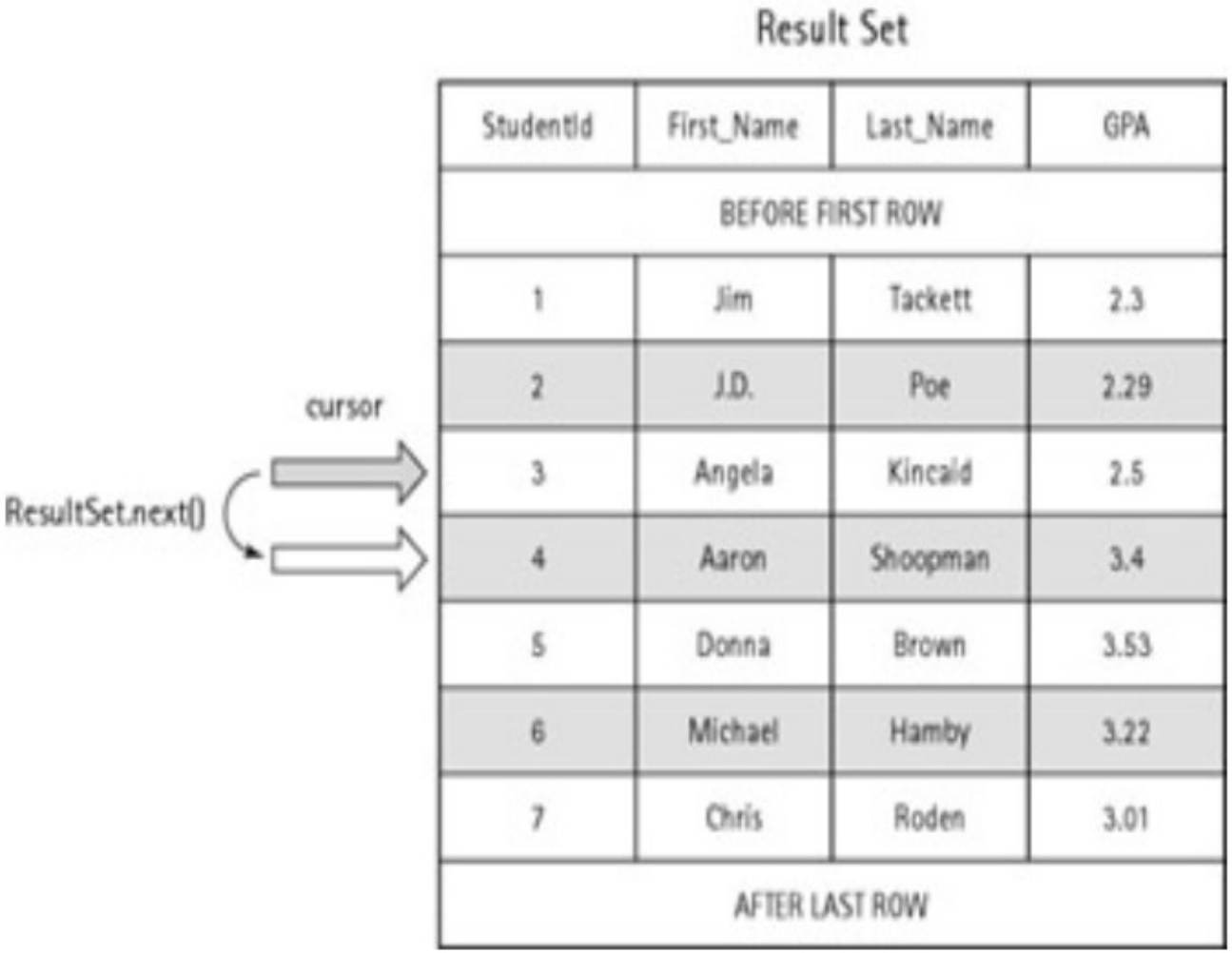


- El cursor puede estar en una fila concreta
- También puede estar en dos filas especiales
  - Antes de la primera fila (*Before the First Row,* BFR)
  - Después de la última fila (*After the Last Row,* ALR)
- Inicialmente el **`ResultSet`** está en BFR
- **`next()`** mueve el cursor hacia delante
  - Devuelve **`true`** si se encuentra en una fila concreta y **`false`** si alcanza el ALR

```
while (rs.next()) {
    String name = rs.getString("titulo");
    float price = rs.getFloat("precio");
    System.out.println(name + "\t" + price);
}
```

Acceso al conjunto de resultados

Posicionamiento del cursor



Acceso al conjunto de resultados

Obtención de los datos de la fila

- Cuando el **`ResultSet`** se encuentra en una fila concreta se pueden usar los métodos de acceso a las columnas
  - **`String getString(String columnLabel)`**
  - **`String getString(int columnIndex)`**
  - **`int getInt(String columnLabel)`**
  - **`int getInt(int columnIndex)`**
  - … (existen dos métodos por cada tipo)

Los índices empiezan en 1 (no en 0)

```
while (rs.next()) {
    String name = rs.getString("titulo");
    float price = rs.getFloat("precio");
    System.out.println(name + "\t" + price);
}
```

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Librerar Recursos

Liberar recursos

- Cuando se termina de usar una **`Connection`**, un **`Statement`** o un **`ResultSet`** es necesario liberar los recursos que necesitan
- Puesto que la información de un **`ResultSet`** no se carga en memoria, existen conexiones de red abiertas
- Métodos **`close()`**:
  - **`ResultSet.close()`** – Libera los recursos del **`ResultSet`**. Se cierran automáticamente al cerrar el Statement que lo creó o al reejecutar el **`Statement`**.
  - **`Statement.close()`** – Libera los recursos del **`Statement`**.
  - **`Connection.close()`** – Finaliza la conexión con la base de datos

Manejar los errores

```
import java.sql.*;
public class HolaMundoBaseDatos {
  public static void main(String[] args)
     throws ClassNotFoundException, SQLException {

        Class.forName("com.mysql.jdbc.Driver");

        Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/sample","root","pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
                "SELECT titulo, precio FROM Libros WHERE precio > 2");

        while (rs.next()) {
                String name = rs.getString("titulo");
                float price = rs.getFloat("precio");
                System.out.println(name + "\t" + price);
        }
        rs.close()
        stmt.close();
        conn.close();
  }
}
```

Manejar los errores

- Hay que gestionar los errores apropiadamente
- Se pueden producir excepciones **`ClassNotFoundException`** si no se encuentra el driver
- Se pueden producir excepciones **`SQLException`** al interactuar con la base de datos
  - SQL mal formado
  - Conexión de red rota
  - Problemas de integridad al insertar datos (claves duplicadas)

```
import java.sql.*;
public class HolaMundoGestionErrores {
  public static void main(String[] args) {

     try {
        Class.forName("com.mysql.jdbc.Driver");
     } catch (ClassNotFoundException e) {
        System.err.println("El driver no se encuentra");
        System.exit(-1);
     }

     Connection conn = null;
     try {
        conn = DriverManager.getConnection(
           "jdbc:mysql://localhost:3306/sample", "root", "pass");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(
              "SELECT titulo, precio FROM Libros WHERE precio > 2");
```

Gestión de errores en la localización del driver

```
        while (rs.next()) {
            String name = rs.getString("titulo");
            float price = rs.getFloat("precio");
            System.out.println(name + "\t" + price);
        }
        rs.close();
        stmt.close();
    } catch (SQLException e) {
        System.err.println("Error en la base de datos: "+
          e.getMessage());
        e.printStackTrace();
    } finally {
        if(conn != null){
            try {
                conn.close();
            } catch (SQLException e) {
                System.err.println("Error al cerrar la conexión: "+
                  e.getMessage());
            }
        }
      }
   }
}
```

Gestión de errores en el envío de consultas

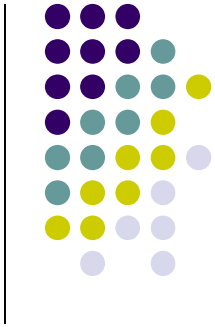
Gestión de errores al cerrar la conexión

Ejercicio 2

- Implementar el ejercicio anterior
- Comprobar la gestión de errores provocando errores
  - Cambia el nombre del driver
  - Cambia el formato de la URL
  - Modifica la sentencia SQL

Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de **`ResultSet`**

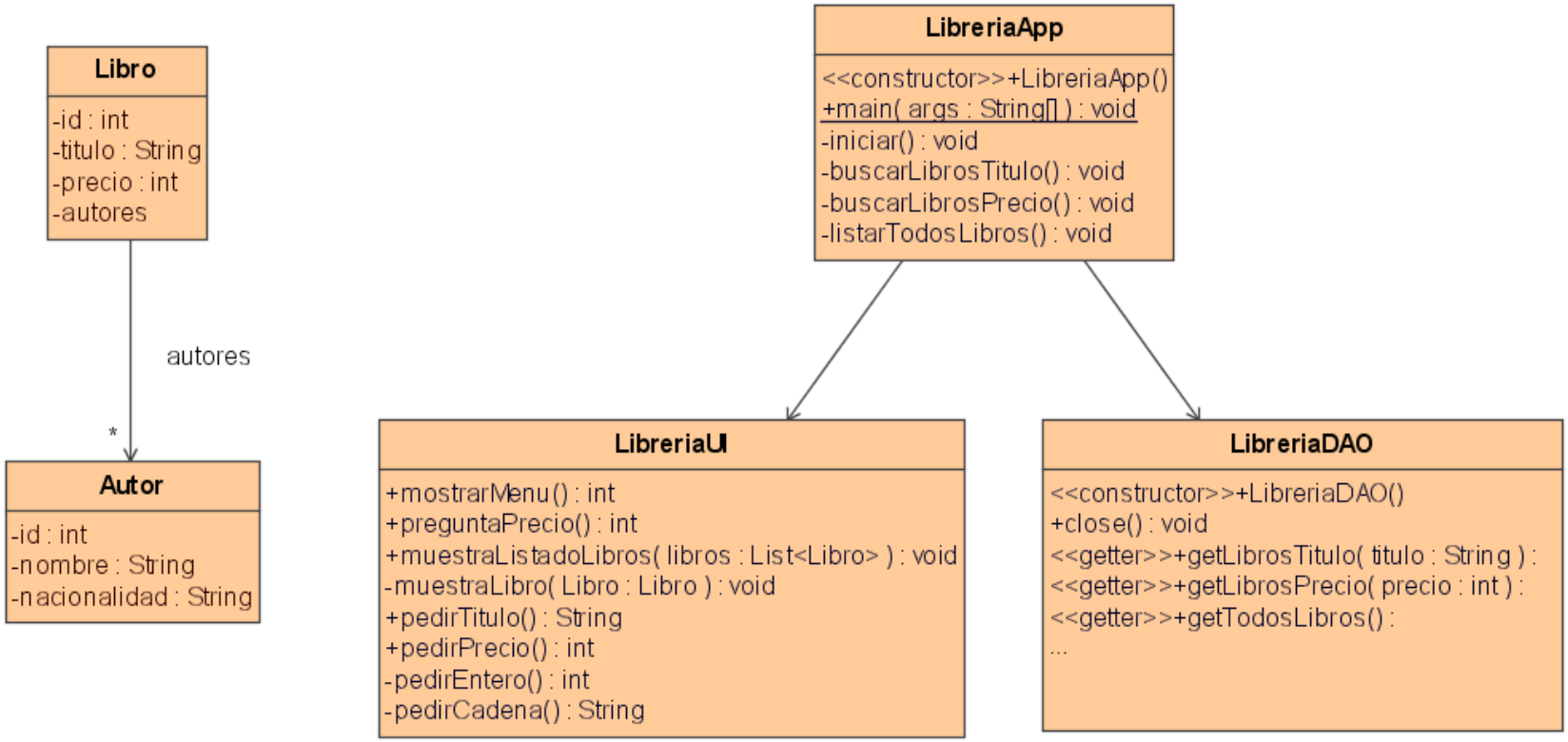
Diseño de una aplicación con BD

- Patrón DAO (*Data Access Object*)
- Cuando se desarrolla una aplicación con BD los detalles de la comunicación con la base de datos se implementan en una clase o módulo
- La información se gestiona como objetos definidos en una clase de Java (clase Libro, Autor…)
- El sistema de persistencia (BD, XML, fichero de texto, servicio web…) se puede cambiar fácilmente
- Se pueden distribuir responsabilidades entre los integrantes del equipo de desarrollo

Ejercicio 3

- Implementar una sencilla aplicación Java que permita gestionar libros y autores en una base de datos
  - Sólo soportará listado de libros
    - Listado de todos los libros
    - Listado por título
    - Listado por precio
  - Al listar los libros deberán mostrarse sus autores

Ejercicio 3


Libro
-id : int
-titulo : String
-precio : int
-autores
autores
*
Autor
-id : int
-nombre : String
-nacionalidad : String
LibreriaApp
<<constructor>>+LibreriaApp()
+main(args : String[]) : void
-iniciar() : void
-buscarLibrosTitulo() : void
-buscarLibrosPrecio() : void
-listarTodosLibros() : void
LibreriaUI
+mostrarMenu() : int
+preguntaPrecio() : int
+muestraListadoLibros(libros : List<Libro>) : void
-muestraLibro(Libro : Libro) : void
+pedirTitulo() : String
+pedirPrecio() : int
-pedirEntero() : int
-pedirCadena() : String
LibreriaDAO
<<constructor>>+LibreriaDAO()
+close() : void
<<getter>>+getLibrosTitulo(titulo : String) :
<<getter>>+getLibrosPrecio(precio : int) :
<<getter>>+getTodosLibros() :
...

Acceso a bases de datos en Java

JDBC

- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de **`ResultSet`**

Conexiones a la base de datos

- Cada objeto **`Connection`** representa una conexión física con la base de datos
- Se pueden especificar más propiedades además del usuario y la password al crear una conexión
- Estas propiedades se pueden especificar:
  - Codificadas en la URL (ver detalles de la base de datos)
  - Usando métodos **`getConnection(…)`** sobrecargados de la clase **`DriverManager`**

Conexiones a la base de datos

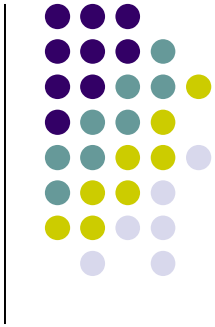
```
String url = "jdbc:mysql://localhost:3306/sample";
String name = "root";
String password = "pass" ;
Connection c = DriverManager.getConnection(url, user, password);
```

```
String url =
   "jdbc:mysql://localhost:3306/sample?user=root&password=pass";
Connection c = DriverManager.getConnection(url);
```

```
String url = "jdbc:mysql://localhost:3306/sample";
Properties prop = new Properties();
prop.setProperty("user", "root");
prop.setProperty("password", "pass");
Connection c = DriverManager.getConnection(url, prop);
```

Acceso a bases de datos en Java

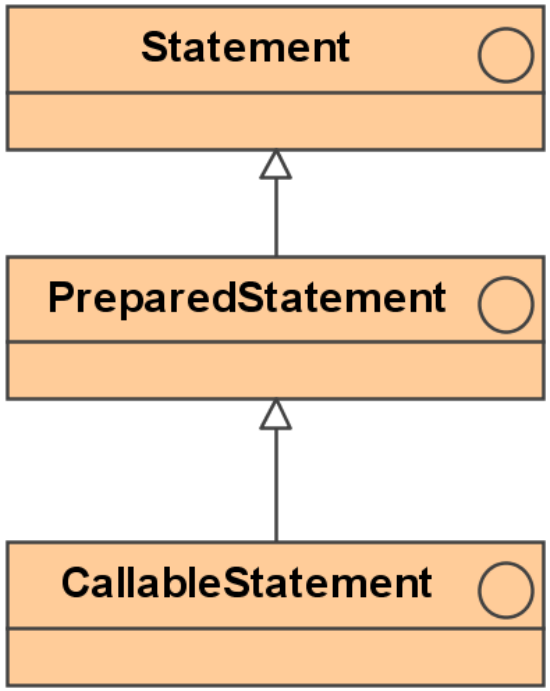
JDBC



- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de **`ResultSet`**

Sentencias SQL

- Con JDBC se pueden usar diferentes tipos de **`Statement`**



SQL estático en tiempo de ejecución, no acepta parámetros

```
Statement stmt = conn.createStatement();
```

Para ejecutar la mismas sentencia muchas veces (la “prepara”). Acepta parámetros

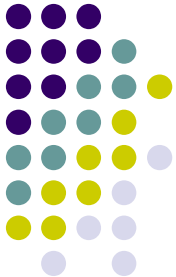
```
PreparedStatement ps =
   conn.prepareStatement(...);
```

Llamadas a procedimientos almacenados

```
CallableStatement s =
   conn.prepareCall(...);
```

Sentencias SQL

Uso de `Statement`



- Tiene diferentes métodos para ejecutar una sentencia
  - **`executeQuery(...)`**
    - Se usa para sentencias SELECT. Devuelve un ResultSet
  - **`executeUpdate(…)`**
    - Se usa para sentencias INSERT, UPDATE, DELETE o sentencias DDL. Devuelve el número de filas afectadas por la sentencia
  - **`execute(…)`**
    - Método genérico de ejecución de consultas. Puede devolver uno o más ResulSet y uno o más contadores de filas afectadas.

Ejercicio 4

- Incorpora a la librería
  - Inserción de libros (con sus autores)

```
INSERT INTO Libros VALUES (1,'Bambi',3)
INSERT INTO Autores VALUES (1,'Pedro','Húngaro')
INSERT INTO relacionlibroautor VALUES (1,1)
```

Ejercicio 4

- Se puede seguir el siguiente esquema para la inserción:
  - Pedir datos libro (también el id)
  - Preguntar número de autores
  - Por cada autor
    - Preguntar si es nuevo
    - Si es nuevo
      - Pedir datos autor (también el id)
      - Insertar autor en base datos (Insertamos datos en la tabla Autores)
      - Guardar Autor en la lista de autores del libro
    - si no
      - Pedir código del autor
      - Cargar el autor
      - Guardar Autor en la lista de autores del libro
  - Insertar nuevo libro (Insertamos datos en las tablas Libros y RelacionLibroAutor)

Sentencias SQL

Uso de `PreparedStatement`

- Los PreparedStatement se utilizan:
  - Cuando se requieren parámetros
  - Cuando se ejecuta muchas veces la misma sentencia
    - La sentencia se prepara al crear el objeto
    - Puede llamarse varias veces a los métodos **`execute`**

```
PreparedStatement ps = conn.
    prepareStatement("INSERT INTO Libros VALUES (?,?,?)");

ps.setInt(1, 23);
ps.setString(2, "Bambi");
ps.setInt(3, 45);

ps.executeUpdate();
```

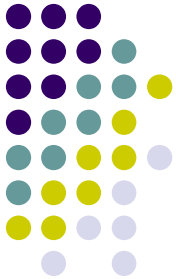
Ejercicio 5

- Incorpora a la librería
  - Borrado de libros (implementado con **`PreparedStatement`**)

```
DELETE FROM relacionlibroautor WHERE idLibro = 1
DELETE FROM libros WHERE idLibro = 1
```

Sentencias SQL

Uso de `CallableStatement`

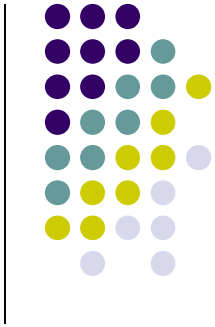


- Permite hacer llamadas a los procedimientos almacenados de la base de datos
- Permite parámetros de entrada IN (como el **`PreparedStatement`**), parámetros de entrada-salida INOUT y parámetros de salida OUT

```
CallableStatement cstmt =
    conn.prepareCall ("{call getEmpName (?,?)}");

cstmt.setInt(1,111111111);
cstmt.registerOutParameter(2,java.sql.Types.VARCHAR);

cstmt.execute();

String empName = cstmt.getString(2);
```

Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de **`ResultSet`**

Transacciones

- Las transacciones tratan un conjunto de sentencias como una única sentencia, de forma que si una falla, todo lo anterior se deshace

```
try {
    conn.setAutoCommit(false);
    Statement statement = conn.createStatement();
    statement.executeUpdate("DELETE …”);
    statement.executeUpdate("DELETE …”);
    conn.commit();
    conn.setAutoCommit(true);
    statement.close();
} catch (SQLException e) {
    try {
        conn.rollback();
    } catch (SQLException e1) {
        System.err.println("Error");
    }
    System.err.println("Error”);
}
```

Por defecto se hace commit por cada sentencia. Hay que desactivarlo

Cuando se han ejecutado todas las sentencias, se hace “commit”

Se vuelve a poner el autocommit, para el resto de la aplicación

Si algo falla, se hace “rollback”

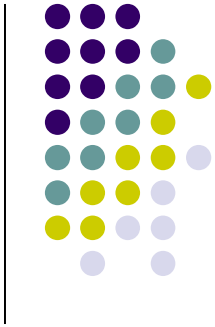
Ejercicio 6

- Incorpora a la librería
  - Borrado de libros con transacciones
  - De esta forma o se asegura de que se borran todas las filas de todas las tablas y no se deja la base de datos inconsistente

```
DELETE FROM relacionlibroautor WHERE idLibro = 1
DELETE FROM libros WHERE idLibro = 1
```

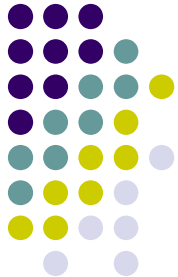
Acceso a bases de datos en Java

JDBC



- Introducción a las bases de datos
- Bases de datos en Java. JDBC
  - Introducción a JDBC
  - Diseño de una aplicación con BD
  - Conexiones a la base de datos
  - Sentencias SQL
  - Transacciones
  - Uso de `ResultSet`

Uso de `ResultSet`



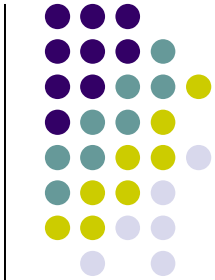
- El **`ResultSet`** es el objeto que representa el resultado de una consulta
- No carga toda la información en memoria
- Se pueden usar para actualizar, borrar e insertar nuevas filas

Uso de `ResultSet`
Características

- Al crear un **`Statement`**, un **`PreparedStatement`** o un **`CallableStatement`**, se pueden configurar aspectos del **`ResultSet`** que devolverá al ejecutar la consulta

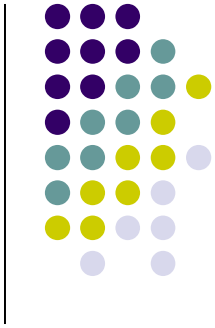
```
createStatement(
    int resultSetType, int resultSetConcurrency);

prepareStatement(String SQL,
    int resultSetType, int resultSetConcurrency);

prepareCall(String sql,
    int resultSetType, int resultSetConcurrency);
```

Uso de `ResultSet`
Características



- Características del **`ResultSet`**
  - resultSetType
    - ResultSet.TYPE_FORWARD_ONLY – Sólo movimiento hacia delante (por defecto)
    - ResultSet.TYPE_SCROLL_INSENSITIVE – Puede hacer cualquier movimiento pero no refleja los cambios en la base de datos
    - ResultSet.TYPE_SCROLL_SENSITIVE – Puede hacer cualquier movimiento y además refleja los cambios en la base de datos
  - resultSetConcurrency
    - ResultSet.CONCUR_READ_ONLY – Sólo lectura (por defecto)
    - ResultSet.CONCUR_UPDATABLE - Actualizable

Uso de `ResultSet`
Características



- Actualización de datos

```
rs.updateString("campo", "valor");
rs.updateInt(1, 3);
rs.updateRow();
```

- Inserción de datos

```
rs.moveToInsertRow();
rs.updateString(1, "AINSWORTH");
rs.updateInt(2,35);
rs.updateBoolean(3, true);
rs.insertRow();

rs.moveToCurrentRow();
```

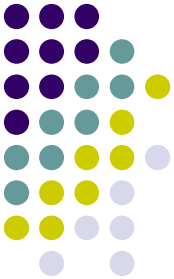
Mueve el cursor a la posición anterior al movimiento a inserción

Ejercicio 7

- Incorpora a la librería
  - Reducción del precio de todos los libros a la mitad de precio
  - Utilizando un **`ResultSet`** actualizable

Uso de `ResultSet`

Posicionamiento del cursor

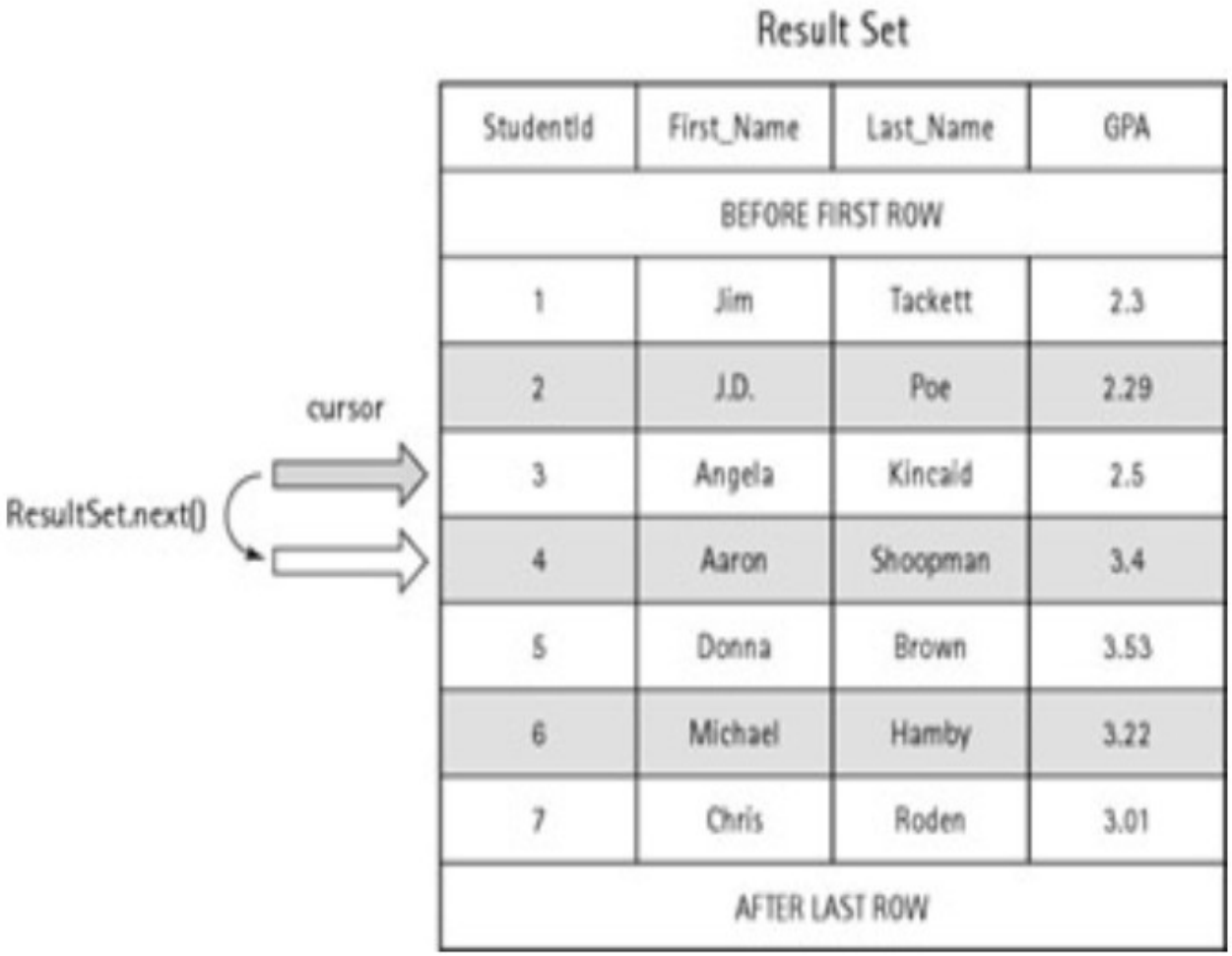


- El cursor puede estar en una fila concreta
- También puede estar en dos filas especiales
  - Antes de la primera fila (*Before the First Row,* BFR)
  - Después de la última fila (*After the Last Row,* ALR)
- Inicialmente el **`ResultSet`** está en BFR
- **`next()`** mueve el cursor hacia delante
  - Devuelve **`true`** si se encuentra en una fila concreta y **`false`** si alcanza el ALR

```
while (rs.next()) {
    String name = rs.getString("titulo");
    float price = rs.getFloat("precio");
    System.out.println(name + "\t" + price);
}
```

Uso de `ResultSet`

Posicionamiento del cursor



Uso de `ResultSet`

- Métodos que permiten un movimiento por el ResultSet
  - next() – Siguiente fila
  - previous() – Fila anterior
  - beforeFirst() – Antes de la primera
  - afterLast() – Después de la última
  - first() – Primera fila
  - last() – Última fila
  - absolute() – Movimiento a una fila concreta
  - relative() – Saltar ciertas filas hacia delante